SOFTWARE ENGINEERING

SOFTWARE DESIGN SPECIFICATION

WORKEX-JOB PORTAL WEB APPLICATION

GROUP MEMBERS:

EBIN ROYCE(IIB2022002)

CECILY AMBOOKEN(IIB2022003)

RISHAB BOHRA(IIB2022004)

VEERATHU SRI SINDHU(IIB2022006)

HARSHIN KAMISHETTY(IIB2022017)

RAKIM MIDDYA(IIT2022255)

THE SOFTWARE DESIGN SPECIFICATION

1. Introduction

The Software Design Document is a document to provide documentation which will be used to aid in software development by providing the details for how the software should be built.

Within the Software Design Document are narrative and graphical documentation of the software design for the project including use case models, sequence diagrams, collaboration models, object behaviour models, and other supporting requirement information.

1.1 Purpose of this document

This document will define the design of the one runway simulator. It contains specific information about the expected input, output, classes, and functions. The interaction between the classes to meet the desired requirements are outlined in detailed figures at the end of the document.

1.2 Scope of the development project

We describe what features are in the scope of the software and what are not in the

scope of the software to be developed.

In Scope:

a. User registeration and authentication.

b. Job searching and filtering.

c. Application submission.

d. Job posting

Out of Scope:

a. Social networking features

b. Search engine optimization (SEO)

1.3 Definitions, acronyms, and abbreviations

IEEE: Institute of Electrical and Electronics Engineers

SDS: Software Design Specification

Req: Request

Res: Response

1.4 References

1.4.1 R. S. Pressman, Software Engineering: A Practioner's Approach, 5th

Ed, McGraw-Hill, 2001.

1.4.2 IEEE SDS template

1.5 Overview of document

This SDS is divided into seven sections with various sub-sections. The sections of the

Software Design Document are:

1. **Introduction:** describes the document, purpose, scope of development project definitions and abbreviations used in the document.

2. **Conceptual Architecture/Architecture Diagram:** describes the overview of components, modules, structure and relationships and user interface issues.

3. **Logical Architecture:** describes Logical Architecture Description and Components.

4. **Execution Architecture:** defines the runtime environment, processes, deployment view.

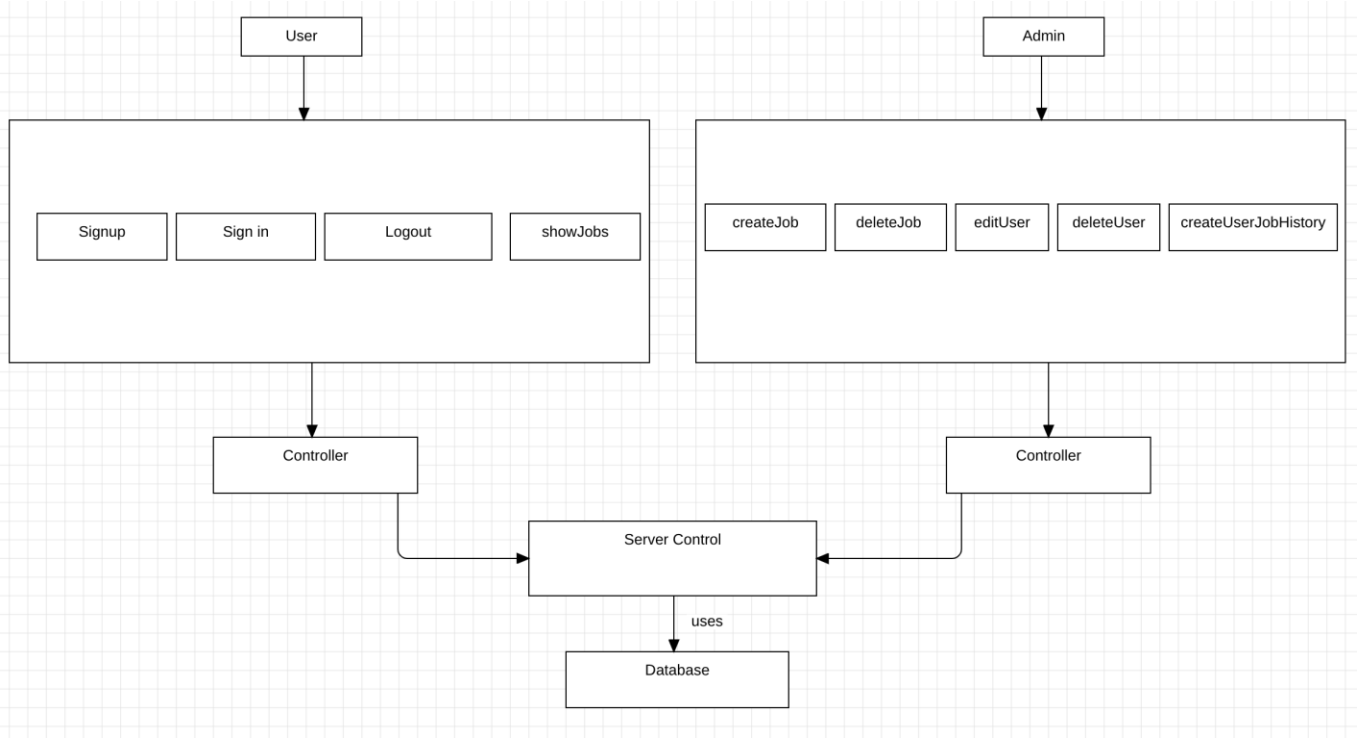
5. **Design Decisions and Trade-offs:** describes the decisions taken along with the reason as to why they were chosen over other alternatives.

6. **Pseudocode for components:** describes pseudocode, as the name indicates.

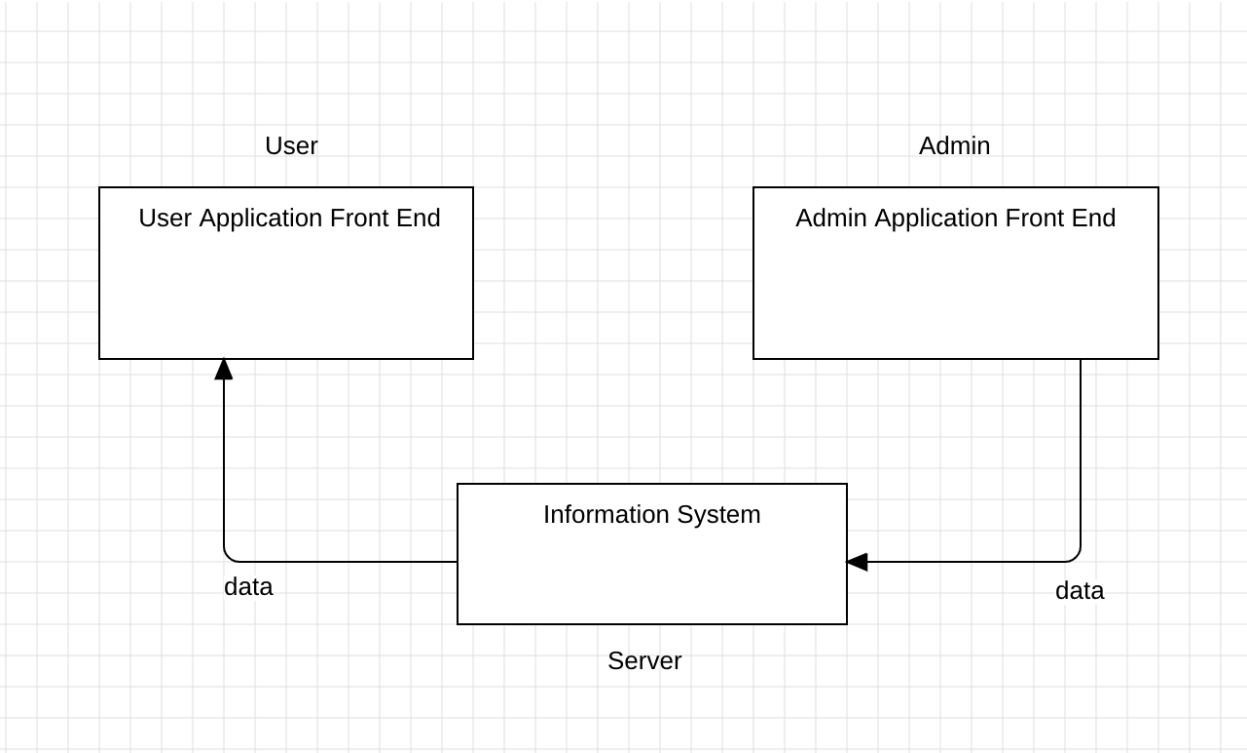
7. **Appendices:** describes subsidiary matter if any.

2. Conceptual Architecture/Architecture Diagram

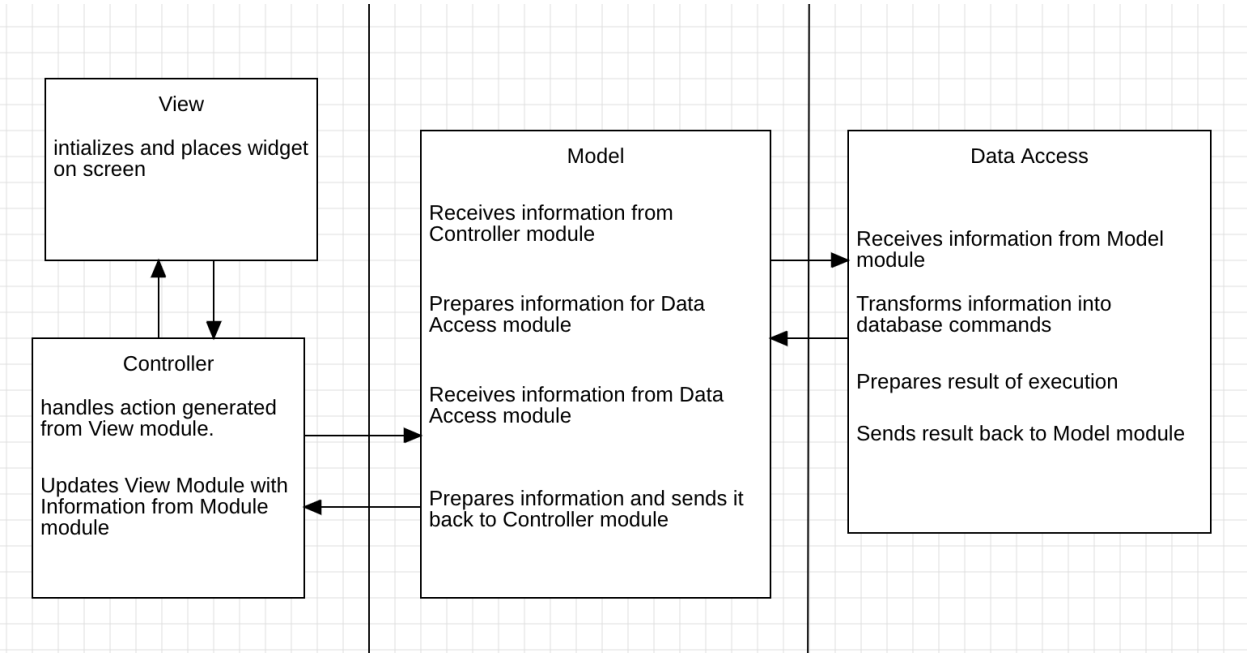
Architecture Diagram 1:



Architecture Diagram 2:



2.1 Overview of modules/components



Note:

The horizontal lines represent the separation of modules.

More than one box within the same section represents sub-modules.

2.2 User Interface Issues

Poor Navigation: Confusing or inconsistent navigation menus can make it difficult for users to find the information they need. For example, if the job search functionality is buried deep within the site or if the menu labels are unclear, users may become frustrated and abandon the site.

Slow Loading Times: Users expect websites to load quickly, and delays can lead to frustration and abandonment. Optimizing page load times by compressing images, minifying code, and leveraging caching techniques can help improve the overall user experience.

Complex Registration Process: Requiring users to fill out lengthy forms or go through multiple steps to create an account can deter potential users from signing up. Simplifying the registration process can help improve user engagement and increase conversion rates.

3. Function Name Descriptions

3.1 Login:

3.1.1 Function: signup

Input:

- Req: Request object containing user's email in the request body.
- Res: Response object.
- Next: Next middleware function.

Output:

- If the email provided already exists in the database: Return an error response with status 400 (Bad Request) indicating "E-mail already registered".
- If there's an error during user creation: Pass the error to the next middleware function.
- If user creation is successful: Return a JSON response with status 201 (Created) containing the newly created user object.

3.1.2 Function: signin

Input:

- Req: Request object containing user's email and password in the request body.
- Res: Response object.
- Next: Next middleware function.

Output:

- If email or password is missing: Return an error response with status 403 (Forbidden) indicating "please add an email" or "please add a password" respectively.
- If user with the provided email doesn't exist: Return an error response with status 400 (Bad Request) indicating "invalid credentials".
- If password provided doesn't match: Return an error response with status 400 (Bad Request) indicating "invalid credentials".
- If sign-in is successful: Call sendTokenResponse function and send token as a cookie in the response.

3.1.3 Function: sendTokenResponse

Input:

- User: User object.

- CodeStatus: HTTP status code.
- Res: Response object.

Output:

- Set a JWT token in a cookie with a max age of 1 hour and HTTP only flag.
- Return a JSON response with status code codeStatus containing success status and user's role.

3.1.4 Function: logout

Input:

- Req: Request object.
- Res: Response object.
- Next: Next middleware function.

Output:

- Clear the token cookie.
- Return a JSON response with status 200 (OK) indicating successful logout.

3.1.5 Function: userProfile

Input:

- Req: Request object containing user's ID.
- Res: Response object.
- Next: Next middleware function.

Output:

- Fetch user profile data by ID and exclude password field.
- Return a JSON response with status 200 (OK) containing success status and user's profile data.

3.1.6 Function: isAuthenticated

Input:

- req: Request object containing cookies.
- res: Response object.
- next: Next middleware function.

Output:

- If token exists in cookies: Proceed to the next middleware function.
- If token doesn't exist: Return an error response with status 401 (Unauthorized) indicating "You must log in!".
- If token verification fails: Return an error response with status 401 (Unauthorized) indicating "You must log in!".

3.1.7 Function: isAdmin

Input:

- req: Request object containing user role information.
- res: Response object.
- next: Next middleware function.

Output:

- If the user role is not 0 (admin): Proceed to the next middleware function.
- If the user role is 0 (not admin): Return an error response with status 401 (Unauthorized) indicating "Access denied, you must be an admin".

3.2 Job Management:

3.2.1 Function: createJob

Input:

- Req: Request object containing job details in the request body and user ID in req.user.id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Create a new job with the provided details and associate it with the user ID.
- Return a JSON response with status 201 (Created) containing success status and the newly created job.

3.2.2 Function: singleJob

Input:

- Req: Request object containing the job ID in req.params.id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a job by ID and populate its jobType field with the jobTypeName.
- Return a JSON response with status 200 (OK) containing success status and the found job.

3.2.3 Function: updateJob

Input:

- Req: Request object containing the job ID in req.params.job_id and updated job details in the request body.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a job by ID, update its details with the provided ones, and populate jobType and user fields.
- Return a JSON response with status 200 (OK) containing success status and the updated job.

3.2.4 Function: deleteJob

Input:

- Req: Request object containing the job ID in req.params.job_id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a job by ID and delete it.
- Return a JSON response with status 200 (OK) containing success status and a message indicating "job deleted".

3.2.5 Function: showJobs

Input:

- Req: Request object containing optional query parameters for filtering, pagination, and search.
- Res: Response object.
- Next: Next middleware function.

Output:

- Enable search functionality based on the keyword provided in the query.
- Filter jobs by category IDs and locations provided in the query.
- Enable pagination for displaying a limited number of jobs per page.
- Fetch jobs based on filters, sort them by creation date, and populate jobType and user fields.
- Return a JSON response with status 200 (OK) containing success status, retrieved jobs, pagination details, and unique locations.

3.3 Job Category:

3.3.1 Function: createJobType

Input:

- Req: Request object containing job type details in the request body and user ID in req.user.id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Create a new job type with the provided details and associate it with the user ID.
- Return a JSON response with status 201 (Created) containing success status and the newly created job type.

3.3.2 Function: alUobsType

Input:

- Req: Request object.
- Res: Response object.
- Next: Next middleware function.

Output:

- Fetch all job types sorted by creation date.
- Return a JSON response with status 200 (OK) containing success status and all job types.

3.3.3 Function: updateJobType

Input:

- Req: Request object containing the job type ID in req.params.type_id and updated job type details in the request body.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a job type by ID, update its details with the provided ones.
- Return a JSON response with status 200 (OK) containing success status and the updated job type.

3.3.4 Function: deleteJobType

Input:

- Req: Request object containing the job type ID in eq.params.type_id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a job type by ID and delete it.
- Return a JSON response with status 200 (OK) containing success status and a message indicating "Job type deleted". If an error occurs during deletion, return a 500 (Internal Server Error) response with an appropriate error message.

3.4 User Management:

3.4.1 Function: allUsers

Input:

- Req: Request object.

- Res: Response object.
- Next: Next middleware function.

Output:

- Enable pagination to limit the number of users displayed per page.
- Fetch all users sorted by creation date and exclude the password field.
- Return a JSON response with status 200 (OK) containing success status, retrieved users, pagination details, and total count of users.

3.4.2 Function: singleUser

Input:

- Req: Request object containing the user ID in req.params.id
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a user by ID.
- Return a JSON response with status 200 (OK) containing success status and the found user.

3.4.3 Function: editUser

Input:

- Req: Request object containing the user ID in req.params.id and updated user details in the request body.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a user by ID, update its details with the provided ones.

- Return a JSON response with status 200 (OK) containing success status and the updated user.

3.4.4 Function: deleteUser

Input:

- Req: Request object containing the user ID in req.params.id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find a user by ID and delete it.
- Return a JSON response with status 200 (OK) containing success status and a message indicating "user deleted".

3.4.5 Function: createUserJobsHistory

Input:

- Req: Request object containing job details in the request body and user ID in req.user._id.
- Res: Response object.
- Next: Next middleware function.

Output:

- Find the current user by ID.
- If the user doesn't exist, return an error response with status 401 (Unauthorized) indicating "You must log in".
- If the user exists, add job history to the user's profile.
- Return a JSON response with status 200 (OK) containing success status and the updated user's profile.

4. Execution Architecture

The runtime environment required is any device with a modern browser and enough bandwidth capacity in order for the website to run smoothly.

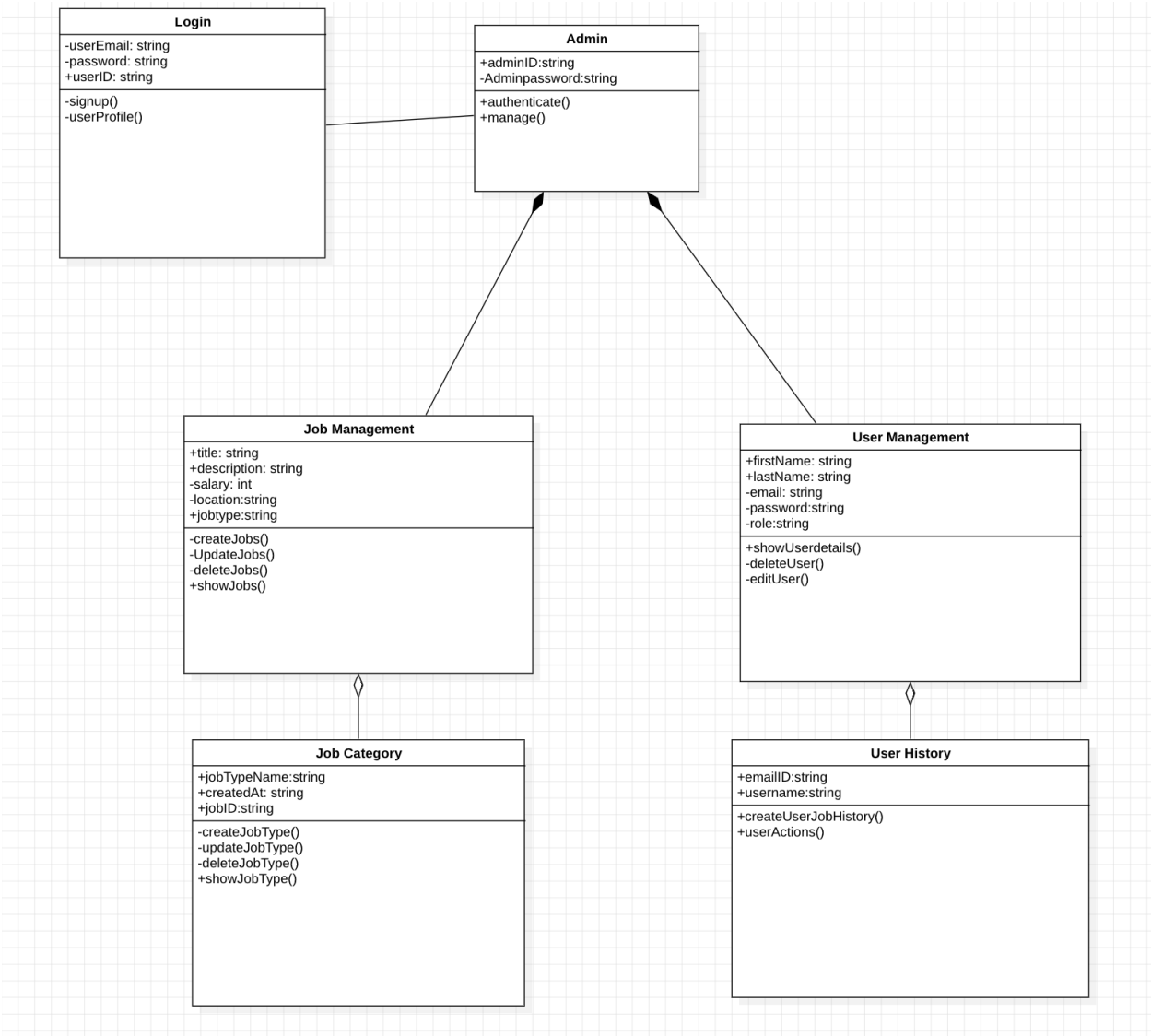
4.1 Reuse and Relationships to other products

No existing or future relationship with other products

5. Logical Architecture(Class,Sequence and State Diagrams)

5.1 Class Diagram

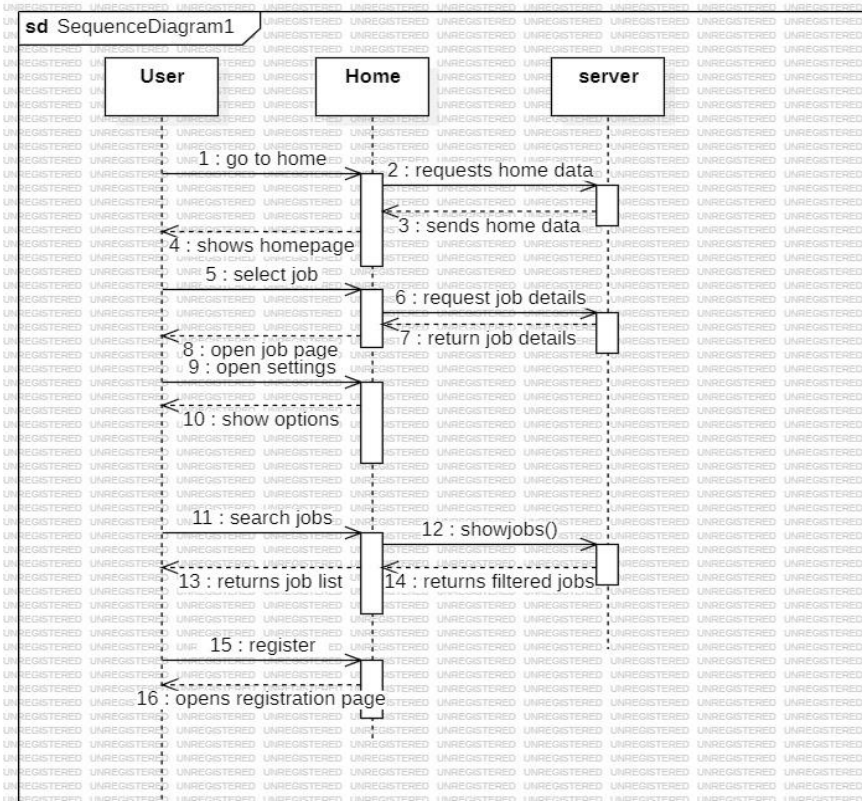
- The classes provided encompass the essential components of a job portal system.
- The "User Management" class represents admin interacting with the system and users and managing job portal.
- The "Job Management" class facilitates job management functionalities, including creating, updating, deleting, and viewing job listings, with attributes like title, description, salary, and location.
- The "Job Category" class handles job categories, allowing for the creation, deletion, and updating of job types.
- Additionally, the "Admin" class, encapsulates administrative functionalities for user management, job type management, and user authorization.
- Classes Job Management and User management are related to admin by Composition.
- User history is related to user management by aggregation .Job category is related to job management by aggregation as well .
- Together, these classes form the backbone of a comprehensive job portal system, providing users with the tools to interact with job listings, manage their profiles, and facilitating administrative tasks for system administrators.



5.2 Sequence Diagram

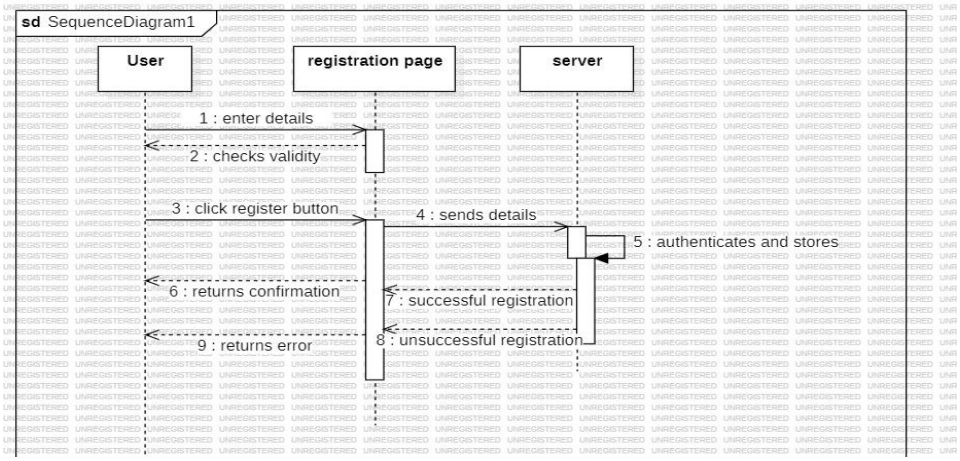
5.2.1 Function: Homepage

User interacts with the server to find a job. Initially unregistered, the user can sign up and then search for openings. They can also view their profile and settings. The user can request job details and see filtered searches based on their input. Finally, the process concludes with the user being registered on the system.



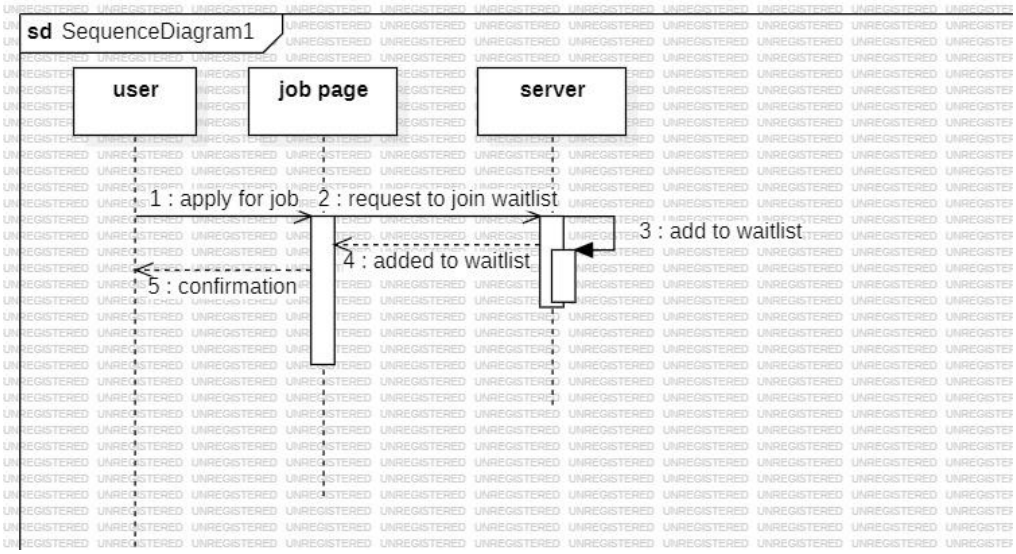
5.2.2 Function: Registeration page

The user can then enter their details on a registration page. The system then checks the validity of the details entered by the user. If the details are valid, the system sends a confirmation message to the user and registers them. If the details are invalid, the system returns an error message to the user.



5.2.3 Function: Job page

If the user finds a job they're interested in, they can apply for it. The server can then add the user to a waitlist for the position or send them a confirmation.

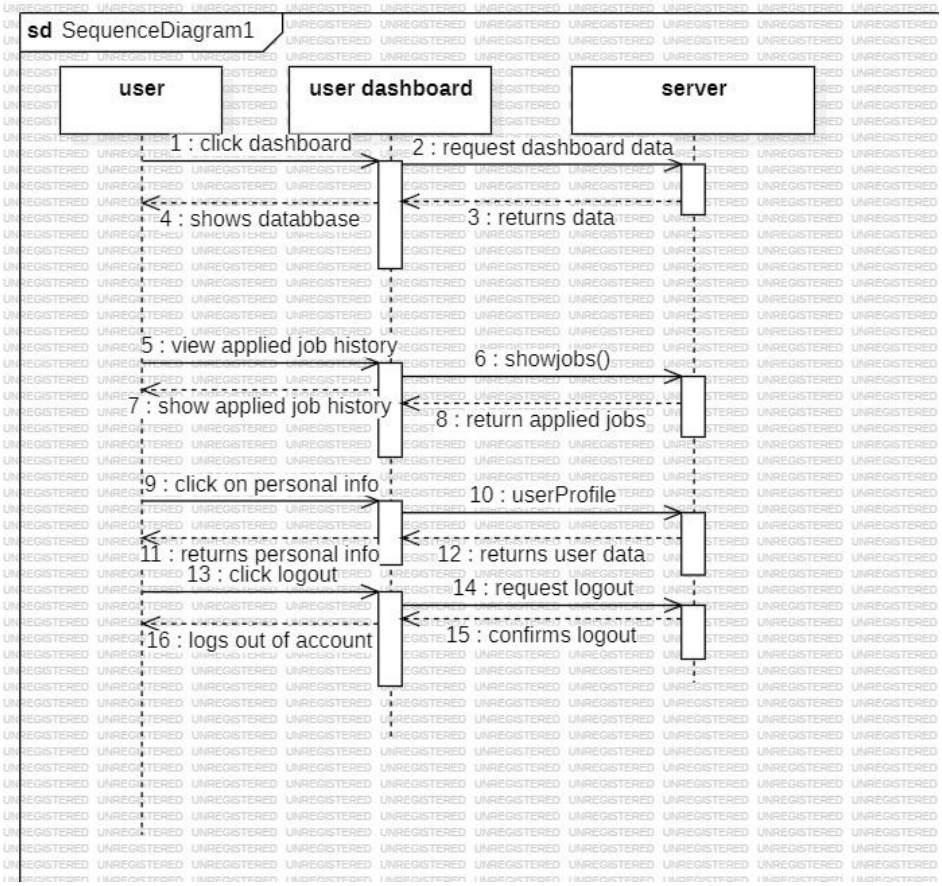


5.2.4 Function: Admin dashboard

- Creating a Job: The process starts with an employer user who is already registered and passes required information through the admin. The admin initiates the creation of a new job by entering the job details. This request is sent to the server. The server creates the job and sends a confirmation back to the employer.
- Editing a Job: The admin can also edit an existing job upon request by the employer. They start by requesting the job details. The server retrieves the details and sends them back. The admin edits the details and sends them back to the server. The server updates the job and sends a confirmation.
- Deleting a Job: If the admin wants to delete a job, they can do so by selecting the delete option. The server acknowledges the request and deletes the job.

5.2.5 Function: User dashboard

- Searching for Jobs: The user can search for jobs by clicking on the dashboard and then viewing the applied job history. The server responds by sending the data requested.
- Viewing User Profile: The user can access their profile information by clicking on it. The server then retrieves and returns the user's data.
- Logging Out: The user can log out of their account by clicking the logout button. The server acknowledges the request and confirms the logout.



5.2.6 Function: Login page

When the user has not logged in yet, the user can then enter their login credentials (username and password) on a login page. These details are then sent to the server. The server authenticates the login credentials.

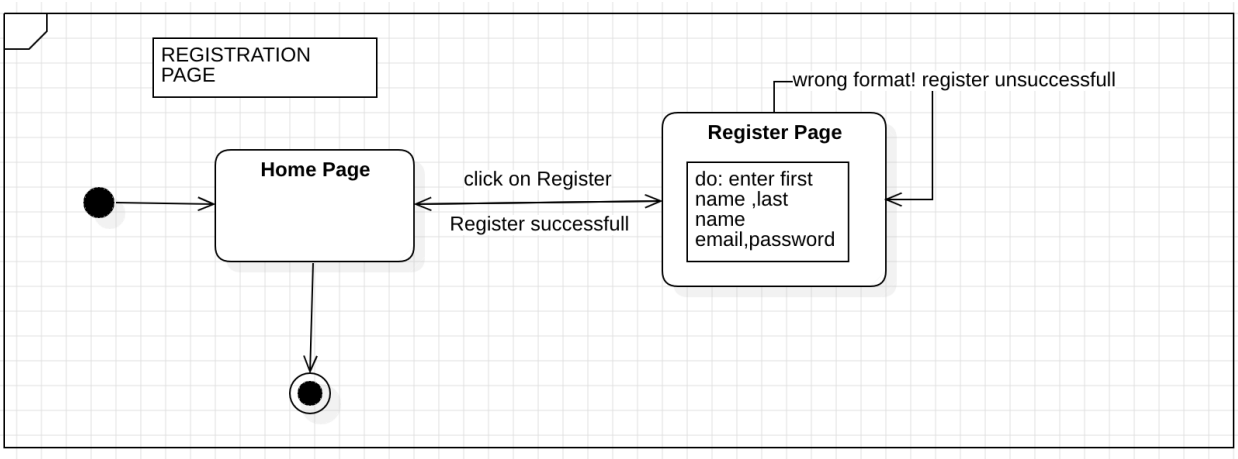
If the credentials are valid, the server sends a successful login message back to the user and the user gains access to the system. If the credentials are invalid, the server sends a failed login message back to the user and the user remains in the unregistered state.

5.3 State Diagram

5.3.1 Function: Registeration Page

The user can then enter their details on a registration page. The system then checks the validity of the details entered by the user. If the details are valid, the system sends a

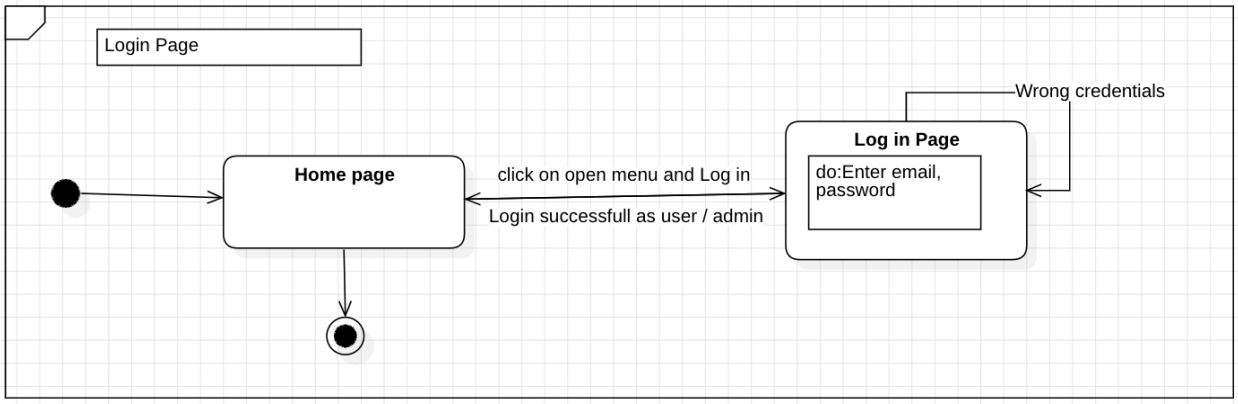
confirmation message to the user and registers them. If the details are invalid, the system returns an error message to the user.



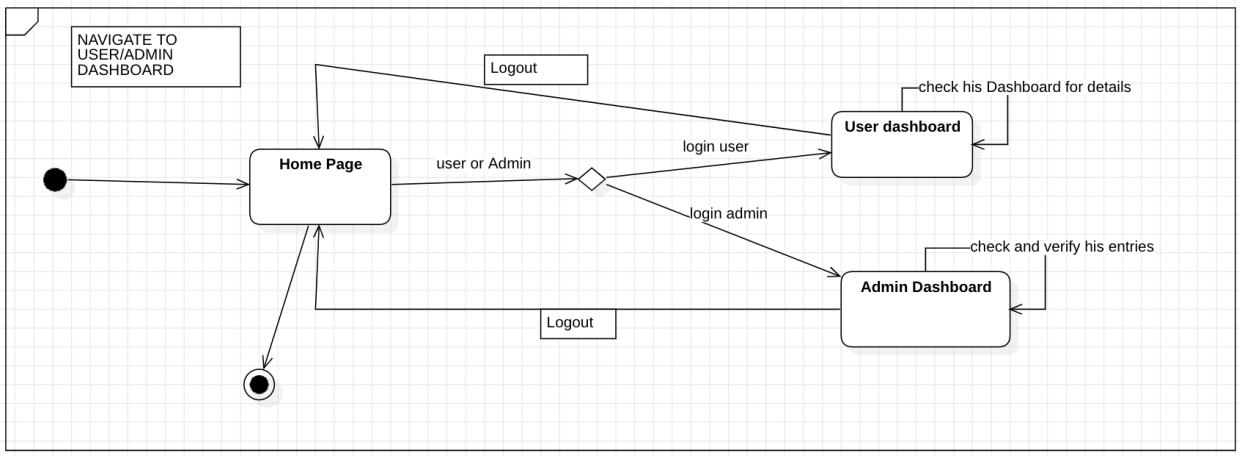
5.3.2 Function: Login Page

When the user has not logged in yet, the user can then enter their login credentials (username and password) on a login page. These details are then sent to the server.

The server authenticates the login credentials. If the credentials are valid, the server sends a successful login message back to the user and the user gains access to the system. If the credentials are invalid, the server sends a failed login message back to the user and the user remains in the unregistered state.

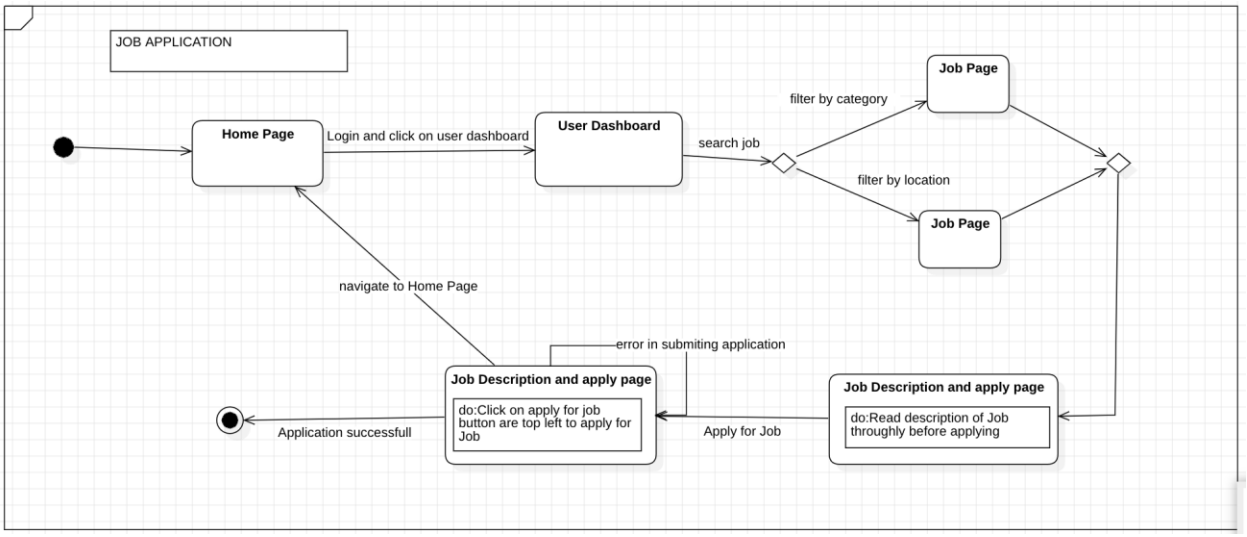


5.3.3 Function: User/Admin Dashboard



5.3.4 Function: Job Application

If the user finds a job they're interested in, they can apply for it. The server can then add the user to a waitlist for the position or send them a confirmation



6. Pseudocode for functions

6.1 Login

6.1.1 Function: signup

```
exports.signup = async (req, res, next) => {
   const { email } = req.body;
   const userExist = await User.findOne({ email });
   if (userExist) {
       return next(new ErrorResponse("E-mail already registred", 400));
   }
   try {
       const user = await User.create(req.body);
       res.status(201).json({
           success: true,
           user
       })
   } catch (error) {
       next(error);
   }
}
```

6.1.2 Function: signin

```
exports.signin = async (req, res, next) => {

   try {
       const { email, password } = req.body;
       //validation
       if (!email) {
           return next(new ErrorResponse("please add an email", 403));
       }
       if (!password) {
           return next(new ErrorResponse("please add a password", 403));
       }

       //check user email
       const user = await User.findOne({ email });
       if (!user) {
           return next(new ErrorResponse("invalid credentials", 400));
       }
       //check password
```

```
        const isMatched = await user.comparePassword(password);
        if (!isMatched) {
            return next(new ErrorResponse("invalid credentials", 400));
        }

        sendTokenResponse(user, 200, res);

    } catch (error) {
        next(error);
    }
}
```

6.1.3 Function: sendTokenResponse

```
const sendTokenResponse = async (user, codeStatus, res) => {
    const token = await user.getJwtToken();
    res
        .status(codeStatus)
        .cookie('token', token, { maxAge: 60 * 60 * 1000, httpOnly:
true })
        .json({
            success: true,
            role: user.role
        })
}
```

6.1.4 Function: logout

```
exports.logout = (req, res, next) => {
    res.clearCookie('token');
    res.status(200).json({
        success: true,
        message: "logged out"
    })
}
```

6.1.5 Function: isAuthenticated

```
exports.isAuthenticated = async (req, res, next) => {
   const { token } = req.cookies;
   // Make sure token exists
   if (!token) {
       return next(new ErrorResponse('You must log in!', 401));
   }

   try {
       // Verify token
       const decoded = jwt.verify(token, process.env.JWT_SECRET);
       req.user = await User.findById(decoded.id);
       next();

   } catch (error) {
       return next(new ErrorResponse('You must log in!', 401));
   }
}
```

6.1.6 Function: isAdmin

```
exports.isAdmin = (req, res, next) => {
   if (req.user.role === 0) {
       return next(new ErrorResponse('Access denied, you must an
admin', 401));
   }
   next();
}
```

6.1.7 Function: userProfile

```
exports.userProfile = async (req, res, next) => {

   const user = await User.findById(req.user.id).select('-
password');

   res.status(200).json({
       success: true,
       user
   })
```

```
}
```

6.2 Job Management

6.2.1 Function: createJob

```
exports.createJob = async (req, res, next) => {
   try {
       const job = await Job.create({
           title: req.body.title,
           description: req.body.description,
           salary: req.body.salary,
           location: req.body.location,
           jobType: req.body.jobType,
           user: req.user.id
       });
       res.status(201).json({
           success: true,
           job
       })
   } catch (error) {
       next(error);
   }
}
```

6.2.2 Function: singleJob

```
exports.singleJob = async (req, res, next) => {
   try {
       const job = await
Job.findById(req.params.id).populate('jobType', 'jobTypeName');
       res.status(200).json({
           success: true,
           job
       })
   } catch (error) {
       next(error);
   }
}
```

6.2.3 Function: updateJob

```
exports.updateJob = async (req, res, next) => {
   try {
       const job = await Job.findByIdAndUpdate(req.params.job_id,
req.body, { new: true }).populate('jobType',
'jobTypeName').populate('user', 'firstName lastName');
       res.status(200).json({
           success: true,
           job
       })
   } catch (error) {
       next(error);
   }
}
```

6.2.4 Function: deleteJob

```
exports.deleteJob = async (req, res, next) => {
   try {
       const job = await Job.findByIdAndDelete(req.params.job_id);
       res.status(200).json({
           success: true,
           message: "job deleted."
       })
   } catch (error) {
       next(error);
   }
}
```

6.2.5 Function: showJobs

```
exports.showJobs = async (req, res, next) => {

   //enable search
   const keyword = req.query.keyword ? {
       title: {
           $regex: req.query.keyword,
           $options: 'i'
```

```
        }
    } : {}
    // filter jobs by category ids
    let ids = [];
    const jobTypeCategory = await JobType.find({}, { _id: 1 });
    jobTypeCategory.forEach(cat => {
        ids.push(cat._id);
    })

    let cat = req.query.cat;
    let categ = cat !== '' ? cat : ids;
    //jobs by location
    let locations = [];
    const jobByLocation = await Job.find({}, { location: 1 });
    jobByLocation.forEach(val => {
        locations.push(val.location);
    });
    let setUniqueLocation = [...new Set(locations)];
    let location = req.query.location;
    let locationFilter = location !== '' ? location :
setUniqueLocation;


    //enable pagination
    const pageSize = 5;
    const page = Number(req.query.pageNumber) || 1;
    //const count = await Job.find({}).estimatedDocumentCount();
    const count = await Job.find({ ...keyword, jobType: categ,
location: locationFilter }).countDocuments();

    try {
        const jobs = await Job.find({ ...keyword, jobType: categ,
location: locationFilter }).sort({ createdAt: -
1 }).populate('jobType', 'jobTypeName').populate('user',
'firstName').skip(pageSize * (page - 1)).limit(pageSize)
        res.status(200).json({
            success: true,
            jobs,
            page,
            pages: Math.ceil(count / pageSize),
            count,
            setUniqueLocation
```

```
        })
    } catch (error) {
        next(error);
    }
}
```

6.3 Job Type Management

6.3.1 Function: createJobType

```
exports.createJobType = async (req, res, next) => {
    try {
        const jobT = await JobType.create({
            jobTypeName: req.body.jobTypeName,
            user: req.user.id,
        });
        res.status(201).json({
            success: true,
            jobT,
        });
    } catch (error) {
        next(error);
    }
};
```

6.3.2 Function: alUobsType

```
exports.allJobsType = async (req, res, next) => {
    try {
        const jobT = await JobType.find().sort({ createdAt: -1 });
        res.status(200).json({
            success: true,
            jobT,
        });
    } catch (error) {
        next(error);
    }
};
```

6.3.3 Function: updateJobType

```
exports.updateJobType = async (req, res, next) => {
   try {
       const jobT = await JobType.findByIdAndUpdate(
           req.params.type_id,
           req.body,
           { new: true }
       );
       res.status(200).json({
           success: true,
           jobT,
       });
   } catch (error) {
       next(error);
   }
};
```

6.3.4 Function: deleteJobType

```
exports.deleteJobType = async (req, res, next) => {
   try {
       const jobT = await
JobType.findByIdAndRemove(req.params.type_id);
       res.status(200).json({
           success: true,
           message: "Job type deleted",
       });
   } catch (error) {
       next(new ErrorResponse("server error", 500));
   }
};
```

6.4 User Management

6.4.1 Function: allUsers

```
exports.allUsers = async (req, res, next) => {
   //enable pagination
   const pageSize = 10;
   const page = Number(req.query.pageNumber) || 1;
   const count = await User.find({}).estimatedDocumentCount();

   try {
       const users = await User.find()
           .sort({ createdAt: -1 })
           .select("-password")
           .skip(pageSize * (page - 1))
           .limit(pageSize);

       res.status(200).json({
           success: true,
           users,
           page,
           pages: Math.ceil(count / pageSize),
           count,
       });
       next();
   } catch (error) {
       return next(error);
   }
};
```

6.4.2 Function: singleUser

```
exports.singleUser = async (req, res, next) => {
   try {
       const user = await User.findById(req.params.id);
       res.status(200).json({
           success: true,
           user,
       });
       next();
```

```
    } catch (error) {
        return next(error);
    }
};
```

6.4.3 Function: editUser

```
exports.editUser = async (req, res, next) => {
    try {
        const user = await User.findByIdAndUpdate(req.params.id,
req.body, {
            new: true,
        });
        res.status(200).json({
            success: true,
            user,
        });
        next();
    } catch (error) {
        return next(error);
    }
};
```

6.4.4 Function: deleteUser

```
exports.deleteUser = async (req, res, next) => {
    try {
        const user = await User.findByIdAndRemove(req.params.id);
        res.status(200).json({
            success: true,
            message: "user deleted",
        });
        next();
    } catch (error) {
        return next(error);
    }
};
```

6.4.5 Function: createUserJobsHistory

```
exports.createUserJobsHistory = async (req, res, next) => {
   const { title, description, salary, location } = req.body;

   try {
       const currentUser = await User.findOne({ _id:
req.user._id });
       if (!currentUser) {
           return next(new ErrorResponse("You must log In", 401));
       } else {
           const addJobHistory = {
               title,
               description,
               salary,
               location,
               user: req.user._id,
           };
           currentUser.jobsHistory.push(addJobHistory);
           await currentUser.save();
       }

       res.status(200).json({
           success: true,
           currentUser,
       });
       next();
   } catch (error) {
       return next(error);
   }
};
```

7. Design Decisions and Tradeoffs

Responsive Design: Ensure the website is optimized for various devices and screen sizes, providing a seamless experience for both desktop and mobile users.

Simplicity: Strive for a clean and intuitive interface that is easy for users to navigate. Avoid overwhelming users with too many options or information on a single page.

Compliance: Ensure compliance with relevant laws and regulations governing job portals, such as employment laws, anti-discrimination laws, and consumer protection regulations.